

1. Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.

Example 1:

Input: words = ["abc","car","ada","racecar","cool"]

Output: "ada"

Explanation: The first string that is palindromic is "ada".

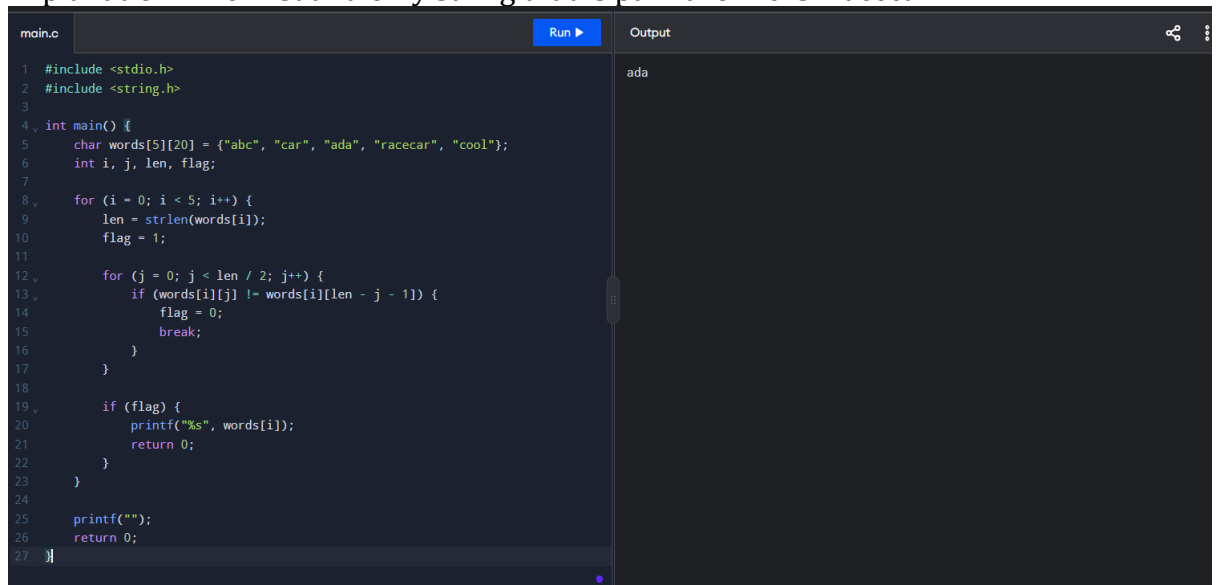
Note that "racecar" is also palindromic, but it is not the first.

Example 2:

Input: words = ["notapalindrome","racecar"]

Output: "racecar"

Explanation: The first and only string that is palindromic is "racecar".



```
main.c
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char words[5][20] = {"abc", "car", "ada", "racecar", "cool"};
6     int i, j, len, flag;
7
8     for (i = 0; i < 5; i++) {
9         len = strlen(words[i]);
10        flag = 1;
11
12        for (j = 0; j < len / 2; j++) {
13            if (words[i][j] != words[i][len - j - 1]) {
14                flag = 0;
15                break;
16            }
17        }
18
19        if (flag) {
20            printf("%s", words[i]);
21            return 0;
22        }
23    }
24
25    printf("");
26    return 0;
27 }
```

Output: ada

2. You are given two integer arrays nums1 and nums2 of sizes n and m, respectively. Calculate the following values: answer1 : the number of indices i such that nums1[i] exists in nums2. answer2 : the number of indices i such that nums2[i] exists in nums1 Return [answer1,answer2].

Example 1:

Input: nums1 = [2,3,2], nums2 = [1,2]

Output: [2,1]

Explanation:

Example 2:

Input: nums1 = [4,3,2,3,1], nums2 = [2,2,5,2,3,6]

Output: [3,4]

Explanation:

The elements at indices 1, 2, and 3 in nums1 exist in nums2 as well. So answer1 is 3.

The elements at indices 0, 1, 3, and 4 in nums2 exist in nums1. So answer2 is 4.

```
main.c Run ▶ Output
1 #include <stdio.h>
2
3 int main() {
4     int a[] = {2,3,2};
5     int b[] = {1,2};
6     int n = 3, m = 2;
7     int i, j, ans1 = 0, ans2 = 0;
8
9     for(i = 0; i < n; i++)
10         for(j = 0; j < m; j++)
11             if(a[i] == b[j]) {
12                 ans1++;
13                 break;
14             }
15
16     for(i = 0; i < m; i++)
17         for(j = 0; j < n; j++)
18             if(b[i] == a[j]) {
19                 ans2++;
20                 break;
21             }
22
23     printf("[%d,%d]", ans1, ans2);
24
25     return 0;
26 }
```

3.You are given a 0-indexed integer array `nums`. The distinct count of a subarray of `nums` is defined as: Let `nums[i..j]` be a subarray of `nums` consisting of all the indices from `i` to `j` such that $0 \leq i \leq j < \text{nums.length}$. Then the number of distinct values in `nums[i..j]` is called the distinct count of `nums[i..j]`. Return the sum of the squares of distinct counts of all subarrays of `nums`. A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [1,2,1]`

Output: 15

Explanation: Six possible subarrays are:

[1]: 1 distinct value

[2]: 1 distinct value

[1]: 1 distinct value

[1,2]: 2 distinct values

[2,1]: 2 distinct values

[1,2,1]: 2 distinct values

The sum of the squares of the distinct counts in all subarrays is equal to $1^2 + 1^2 + 1^2 + 2^2 + 2^2 + 2^2 = 15$.

Example 2:

Input: `nums = [1,1]`

Output: 3

Explanation: Three possible subarrays are:

[1]: 1 distinct value

[1]: 1 distinct value

[1,1]: 1 distinct value

The sum of the squares of the distinct counts in all subarrays is equal to $1^2 + 1^2 + 1^2 = 3$.

```
main.c Run ▶ Output
1 #include <stdio.h>
2
3 int main() {
4     int a[] = {1, 2, 1};
5     int n = 3, sum = 0;
6     int i, j, k, count;
7
8     for(i = 0; i < n; i++) {
9         int distinct[10] = {0};
10
11         for(j = i; j < n; j++) {
12             distinct[a[j]] = 1;
13             count = 0;
14
15             for(k = 0; k < 10; k++)
16                 count += distinct[k];
17
18             sum += count * count;
19         }
20     }
21
22     printf("%d", sum);
23
24     return 0;
25 }
```

15

4. Given a 0-indexed integer array `nums` of length `n` and an integer `k`, return *the number of pairs (i, j) where $0 \leq i < j < n$, such that `nums[i] == nums[j]` and $(i * j)$ is divisible by `k`.*

Example 1:

Input: `nums = [3,1,2,2,2,1,3]`, `k = 2`

Output: 4

Explanation:

There are 4 pairs that meet all the requirements:

- `nums[0] == nums[6]`, and $0 * 6 == 0$, which is divisible by 2.
- `nums[2] == nums[3]`, and $2 * 3 == 6$, which is divisible by 2.
- `nums[2] == nums[4]`, and $2 * 4 == 8$, which is divisible by 2.
- `nums[3] == nums[4]`, and $3 * 4 == 12$, which is divisible by 2.

Example 2:

Input: `nums = [1,2,3,4]`, `k = 1`

Output: 0

Explanation: Since no value in `nums` is repeated, there are no pairs (i,j) that meet all the requirements.

```
main.c Run ▶ Output
1 #include <stdio.h>
2
3 int main() {
4     int nums[] = {3,1,2,2,2,1,3};
5     int n = 7, k = 2, count = 0;
6     int i, j;
7
8     for(i = 0; i < n; i++) {
9         for(j = i + 1; j < n; j++) {
10             if(nums[i] == nums[j] && (i * j) % k == 0)
11                 count++;
12         }
13     }
14
15     printf("%d", count);
16
17     return 0;
18 }
```

4

/www.programiz.com

5. Write a program FOR THE BELOW TEST CASES with least time complexity

Test Cases: -

Input: {1, 2, 3, 4, 5} Expected Output: 5

Input: {7, 7, 7, 7, 7} Expected Output: 7

Input: {-10, 2, 3, -4, 5} Expected Output: 5

<pre>main.c 1 #include <stdio.h> 2 3 int main() { 4 int a[] = {1, 2, 3, 4, 5}; 5 int n = 5, max = a[0], i; 6 7 for(i = 1; i < n; i++) { 8 if(a[i] > max) 9 max = a[i]; 10 } 11 12 printf("%d", max); 13 14 return 0; 15 }</pre>	Run ▶	Output	5
<pre>main.c 1 #include <stdio.h> 2 3 int main() { 4 int a[] = {7, 7, 7, 7, 7}; 5 int n = 5, max = a[0], i; 6 7 for(i = 1; i < n; i++) { 8 if(a[i] > max) 9 max = a[i]; 10 } 11 12 printf("%d", max); 13 14 return 0; 15 }</pre>	Run ▶	Output	7
<pre>main.c 1 #include <stdio.h> 2 3 int main() { 4 int a[] = {-10, 2, 3, -4, 5}; 5 int n = 5, max = a[0], i; 6 7 for(i = 1; i < n; i++) { 8 if(a[i] > max) 9 max = a[i]; 10 } 11 12 printf("%d", max); 13 14 return 0; 15 }</pre>	Run ▶	Output	5

6. You have an algorithm that process a list of numbers. It firstly sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

Test Cases

1. Empty List

1. Input: []

2. Expected Output: None or an appropriate message indicating that the list is empty.

2. Single Element List

1. Input: [5]

2. Expected Output: 5

3. All Elements are the Same

1. Input: [3, 3, 3, 3, 3]

2. Expected Output: 3

main.c	Run ▶	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a[] = {3, 3, 3, 3, 3}; 5 int n = 5, i, j, temp; 6 7 if(n == 0) { 8 printf("List is empty"); 9 return 0; 10 } 11 12 // Sorting 13 for(i = 0; i < n - 1; i++) { 14 for(j = i + 1; j < n; j++) { 15 if(a[i] > a[j]) { 16 temp = a[i]; 17 a[i] = a[j]; 18 a[j] = temp; 19 } 20 } 21 } 22 23 // Maximum element 24 printf("%d", a[n - 1]); 25 26 return 0; 27 }</pre>		3
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a[] = {5}; 5 int n = 5, i, j, temp; 6 7 if(n == 0) { 8 printf("List is empty"); 9 return 0; 10 } 11 12 // Sorting 13 for(i = 0; i < n - 1; i++) { 14 for(j = i + 1; j < n; j++) { 15 if(a[i] > a[j]) { 16 temp = a[i]; 17 a[i] = a[j]; 18 a[j] = temp; 19 } 20 } 21 } 22 23 // Maximum element 24 printf("%d", a[n - 1]); 25 26 return 0; 27 }</pre>		3
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a[] = {3, 3, 3, 3, 3, 3}; 5 int n = 5, i, j, temp; 6 7 if(n == 0) { 8 printf("List is empty"); 9 return 0; 10 } 11 12 // Sorting 13 for(i = 0; i < n - 1; i++) { 14 for(j = i + 1; j < n; j++) { 15 if(a[i] > a[j]) { 16 temp = a[i]; 17 a[i] = a[j]; 18 a[j] = temp; 19 } 20 } 21 } 22 23 // Maximum element 24 printf("%d", a[n - 1]); 25 26 return 0; 27 }</pre>		3

7. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

Test Cases

Some Duplicate Elements

Input: [3, 7, 3, 5, 2, 5, 9, 2]

Expected Output: [3, 7, 5, 2, 9] (Order may vary based on the algorithm used)

Negative and Positive Numbers

Input: [-1, 2, -1, 3, 2, -2]

Expected Output: [-1, 2, 3, -2] (Order may vary)

List with Large Numbers

Input: [1000000, 999999, 1000000]

Expected Output: [1000000, 999999]

```
#include <stdio.h>

int main() {
    int a[] = {3, 7, 3, 5, 2, 5, 9, 2};
    int n = 8, i, j, k = 0, found;

    int unique[8];

    for(i = 0; i < n; i++) {
        found = 0;

        for(j = 0; j < k; j++) {
            if(a[i] == unique[j]) {
                found = 1;
                break;
            }
        }

        if(!found)
            unique[k++] = a[i];
    }

    printf("\n");
    for(i = 0; i < k; i++) {
        printf("%d", unique[i]);
        if(i < k - 1)
            printf(" ");
    }
    printf("\n");

    return 0;
}
```

Output: 3, 7, 5, 2, 9

```
#include <stdio.h>

int main() {
    int a[] = {-1, 2, -1, 3, 2, -2};
    int n = 6, i, j, k = 0, found;

    int unique[6];

    for(i = 0; i < n; i++) {
        found = 0;

        for(j = 0; j < k; j++) {
            if(a[i] == unique[j]) {
                found = 1;
                break;
            }
        }

        if(!found)
            unique[k++] = a[i];
    }

    printf("\n");
    for(i = 0; i < k; i++) {
        printf("%d", unique[i]);
        if(i < k - 1)
            printf(" ");
    }
    printf("\n");

    return 0;
}
```

Output: -1, 2, 3, -2, -456138896, 8

```
#include <stdio.h>

int main() {
    int a[] = {1000000, 999999, 1000000};
    int n = 3, i, j, k = 0, found;

    int unique[3];

    for(i = 0; i < n; i++) {
        found = 0;

        for(j = 0; j < k; j++) {
            if(a[i] == unique[j]) {
                found = 1;
                break;
            }
        }

        if(!found)
            unique[k++] = a[i];
    }

    printf("\n");
    for(i = 0; i < k; i++) {
        printf("%d", unique[i]);
        if(i < k - 1)
            printf(" ");
    }
    printf("\n");

    return 0;
}
```

Output: 1000000, 999999, 0, 0, 5, 5, 7

8. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

```
main.c Run Output
#include <stdio.h>

int main() {
    int a[] = {5, 3, 8, 1, 2};
    int n = 5, i, j, temp;

    for(i = 0; i < n - 1; i++) {
        for(j = 0; j < n - i - 1; j++) {
            if(a[j] > a[j+1]) {
                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
    }

    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

1 2 3 5 8

9. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

Test Case:

Example X={ 3,4,6,-9,10,8,9,30} KEY=10

Output: Element 10 is found at position 5

Example X={ 3,4,6,-9,10,8,9,30} KEY=100

Output : Element 100 is not found

```
main.c Run Output
#include <stdio.h>

int main() {
    int a[] = {3,4,6,-9,10,8,9,30};
    int n = 8, key = 10;
    int i, j, temp, low, high, mid, found = 0;

    // Sort array
    for(i = 0; i < n-1; i++)
        for(j = i+1; j < n; j++)
            if(a[i] > a[j]) {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }

    // Binary Search
    low = 0;
    high = n-1;
    while(low <= high) {
        mid = (low + high) / 2;
        if(a[mid] == key) {
            printf("Element %d is found at position %d", key, mid + 1);
            found = 1;
            break;
        }
        else if(a[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    if(!found)
        printf("Element %d is not found", key);

    return 0;
}
```

Element 10 is found at position 7

10. Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n \log(n))$ time complexity and with the smallest space complexity possible.

```
main.c Run Output
#include <stdio.h>

void merge(int a[], int l, int m, int r) {
    int i = l, j = m + 1, k = 0;
    int temp[r - l + 1];

    while(i <= m && j <= r) {
        if(a[i] < a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }

    while(i <= m)
        temp[k++] = a[i++];
    while(j <= r)
        temp[k++] = a[j++];

    for(i = l, k = 0; i <= r; i++)
        a[i] = temp[k++];
}

void mergeSort(int a[], int l, int r) {
    if(l < r) {
        int m = (l + r) / 2;
        mergeSort(a, l, m);
        mergeSort(a, m + 1, r);
        merge(a, l, m, r);
    }
}

int main() {
    int a[] = {5, 2, 8, 1, 3};
    int n = 5, i;

    mergeSort(a, 0, n - 1);

    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

1 2 3 5 8